

Deutsch in Qiskit, and Real Hardware

3, 5 November · Objectives 5, 6

Building the four oracles, running the algorithm, and then running it somewhere the physics fights back. The gap between simulator and device is the actual subject of this week.

The four oracles

Each is a reversible circuit implementing $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$:

```
from qiskit import QuantumCircuit

def oracle(kind):
    """Two qubits: 0 = input x, 1 = ancilla y."""
    qc = QuantumCircuit(2, name=f"U_f[{kind}]")
    if kind == "constant_0":
        pass # f(x) = 0: identity, nothing to XOR
    elif kind == "constant_1":
        qc.x(1) # f(x) = 1: flip ancilla unconditionally
    elif kind == "balanced_id":
        qc.cx(0, 1) # f(x) = x: XOR input onto ancilla
    elif kind == "balanced_not":
        qc.cx(0, 1); qc.x(1) # f(x) = NOT x
    return qc
```

Note that the two constant oracles never touch qubit 0. Whatever the algorithm learns, it learns from a gate that does not act on the register being measured — which is phase kickback stated as an observation about the circuit diagram.

The algorithm

```
def deutsch(kind):
    qc = QuantumCircuit(2, 1)

    qc.x(1); qc.h(1)          # ancilla -> |-), arming phase kickback
    qc.h(0)                   # input   -> |+), both branches live

    qc.compose(oracle(kind), inplace=True)

    qc.h(0)                   # interfere: relative phase -> amplitude
    qc.measure(0, 0)
    return qc

from qiskit_aer import AerSimulator
from qiskit import transpile

sim = AerSimulator()
for kind in ["constant_0", "constant_1", "balanced_id", "balanced_not"]:
    qc = deutsch(kind)
    counts = sim.run(transpile(qc, sim), shots=1024).result().get_counts()
    verdict = "constant" if max(counts, key=counts.get) == "0" else "balanced"
    print(f"{kind:<14} {counts} -> {verdict}")
```

```
constant_0      {'0': 1024} -> constant
constant_1      {'0': 1024} -> constant
balanced_id     {'1': 1024} -> balanced
balanced_not    {'1': 1024} -> balanced
```

Deterministic — every shot agrees. Interference has made the answer certain, not merely likely. Any deviation on hardware is therefore pure error, which makes this circuit an unusually clean diagnostic.

Verifying the phase

Confirm the mechanism rather than trusting the output:

```
from qiskit.quantum_info import Statevector

qc = QuantumCircuit(2)
qc.x(1); qc.h(1); qc.h(0)
qc.compose(oracle("balanced_id"), inplace=True)
print(Statevector(qc).draw('latex_source'))
# (|0⟩ - |1⟩)/√2 ⊗ |-⟩ - the minus sign IS the answer, before the final H
```

The ancilla returns unchanged in $|-\rangle$ while the input register acquired a relative phase. Nothing was written to the ancilla; the function's value lives entirely in a sign on the register that was never the gate's target.

Running on hardware

```
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2

service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)
print(f"{backend.name}: {backend.num_qubits} qubits")

qc = deutsch("balanced_id")

# optimization_level=3 is not optional on hardware – it routes the circuit
# onto physically coupled qubits and cancels redundant gates.
qc_hw = transpile(qc, backend, optimization_level=3)
print(f"depth {qc.depth()} -> {qc_hw.depth()}, ops {qc_hw.count_ops()}")

job = SamplerV2(mode=backend).run([qc_hw], shots=4096)
print(job.result()[0].data.c.get_counts())
# {'1': 3894, '0': 202} – ~5% error on a 4-gate circuit
```

Five percent wrong on a circuit this shallow is the number to sit with. The theoretical answer is deterministic; the device disagrees one time in twenty.

Where the error comes from

Budget it explicitly:

SOURCE	TYPICAL RATE	NOTES
Single-qubit gate	1e-4 – 1e-3	usually negligible here
Two-qubit gate (CX)	3e-3 – 1e-2	dominates gate error
Readout	1e-2 – 3e-2	often the largest term
T1 (amplitude damping)	100-300 μ s	1> decays toward 0>
T2 (dephasing)	80-200 μ s	relative phase randomizes

For Deutsch, with one CX and one measurement, **readout error dominates**. T2 matters more than T1 in principle, because the algorithm stores its answer in a *phase* — though at ~1 μ s circuit duration against ~100 μ s T2, decoherence contributes little here. On deeper circuits that ordering reverses.

Analogy boundary — decoherence as bit rot. Data degrading in storage is the right intuition for T1: $|1\rangle$ relaxes toward $|0\rangle$, an asymmetric, energy-losing process much like a biased corruption. T2 has no classical counterpart. **Dephasing destroys relative phase while leaving every measurement probability in the computational basis unchanged.** Your monitoring would report the data intact — populations correct, checksums matching in that basis — while the quantity the algorithm depends on has silently vanished. Detecting it requires measuring in a rotated basis, which is exactly what randomized benchmarking does.

Measurement error mitigation

Readout error is the cheapest to correct, because it is characterized independently of the circuit:

```
# Calibrate: prepare each basis state, measure, build the confusion matrix.
cal = []
for bits in ["0", "1"]:
    c = QuantumCircuit(1, 1)
    if bits == "1": c.x(0)
    c.measure(0, 0)
    cal.append(transpile(c, backend, optimization_level=3))

# The result is P(measured | prepared). Inverting it and applying to your
# raw counts recovers an estimate of the true distribution.
```

Qiskit Runtime applies this automatically via `resilience_level=1`. Note the limit: mitigation is post-processing on the *distribution*, so it improves estimates while amplifying statistical noise, and it does nothing for gate error. It is not error correction — that is Week 16, and it is a different kind of thing entirely.

Key takeaways

- The constant oracles never act on the measured register; the answer arrives entirely by phase kickback.
- Deutsch's output is deterministic in theory, which makes any hardware deviation a direct error measurement.
- `optimization_level=3` is mandatory on hardware — routing and gate cancellation change depth substantially.
- On shallow circuits readout error usually dominates; two-qubit gate error dominates as depth grows.

- T2 dephasing destroys phase while leaving computational-basis statistics intact — invisible to the obvious diagnostic.

Self-check

Q1 (*Objective 5*) — Your Deutsch circuit returns 95% '1' on hardware and 100% on the simulator. Your colleague proposes averaging more shots. Explain why this cannot work, and give what would.

Q2 (*Objectives 5, 6*) — A device reports $T_1 = 200 \mu\text{s}$, $T_2 = 120 \mu\text{s}$, and your circuit runs in $2 \mu\text{s}$. Estimate the decoherence contribution, and explain why $T_2 < T_1$ always holds.

Check your answers

A1 — More shots cannot help because the error is **systematic, not statistical**. Additional sampling reduces the standard error of an estimate around whatever value the device actually produces — here ≈ 0.95 . It converges more precisely on the wrong number. Statistical error already sits near $\pm 0.7\%$ at 4096 shots, far smaller than the 5% bias.

What would work, in ascending cost:

Readout mitigation. Characterize the confusion matrix from calibration circuits and invert it. If readout is the dominant term, this recovers most of the gap for a few extra circuits.

`resilience_level=1` in Qiskit Runtime does exactly this.

Better qubit selection. Device error rates vary substantially across qubits. Query `backend.properties()` for per-qubit readout error and per-pair CX fidelity and pin the circuit to the best ones with `initial_layout`.

Dynamical decoupling. Insert pulse sequences during idle periods to refocus dephasing. Available through the transpiler's scheduling passes; helps most on circuits with idle time.

Zero-noise extrapolation. Run at deliberately amplified noise (stretched gates or inserted CX pairs), fit the trend, extrapolate to zero. This costs several times the circuit executions and is what `resilience_level=2` provides.

The general principle: statistical error scales as $1/\sqrt{N}$ and systematic error does not scale with N at all. Diagnose which you have before spending anything.

A2 — Decoherence during the circuit is governed by the ratio of duration to coherence time:

phase coherence: $\exp(-t/T_2) = \exp(-2/120) \approx 0.983 \rightarrow \sim 1.7\% \text{ loss}$
amplitude decay: $\exp(-t/T_1) = \exp(-2/200) \approx 0.990 \rightarrow \sim 1.0\% \text{ loss}$

Roughly 2–3% combined — a meaningful fraction of the observed 5%, though readout error at 1–3% is likely the larger single contributor. Since Deutsch's answer is stored in a relative phase, the T_2 term is the one that directly threatens correctness.

Why $T_2 \leq 2 \cdot T_1$ always. T_1 is energy relaxation — the $|1\rangle$ population decaying to $|0\rangle$. T_2 is the loss of phase coherence, and it has two independent causes: energy relaxation (any T_1 event destroys phase, since the state is gone) plus **pure dephasing** T_ϕ from noise that randomizes phase without exchanging energy. They combine as:

$$1/T_2 = 1/(2 \cdot T_1) + 1/T_\phi$$

Since $1/T_\phi \geq 0$, we get $T_2 \leq 2 \cdot T_1$. The pure-dephasing channel is usually dominant in superconducting qubits — flux noise, charge noise, and thermal photons in the readout resonator all randomize phase without causing a transition. That is why $T_2 = 120 \mu\text{s}$ sits well below the $2 \cdot T_1 = 400 \mu\text{s}$ ceiling, and why dynamical decoupling, which targets T_ϕ specifically, is worth the trouble.